Sprint 2 · Seguridad con NULL y primer enlace entre structs

Objetivo del sprint	Proteger las funciones contra NULL y apuntadores repetidos, intercambiar apuntadores y empezar a conectar structs creando `Cancion` y asignándola a una `Idol`.
Etapas que cubre	Etapa 3 y primera parte de la Etapa 4
Puntos comprometidos	16 puntos en 6 historias
Incremento esperado	`manager.c` con la Etapa 3 completa, y con `Cancion`, el campo `cancion` y `asignar_cancion` funcionando.

Sprint Backlog

ID	Historia	Tareas sugeridas	Pts	Responsable
HU-09	Colaboración segura entre dos integrantes	Escribir `colaboracion` con validaciones · Probar `(&jennie, &jennie)` y `(NULL, &rose)` · Probar un caso válido	3	3
HU-10	Dinámica por turnos con intercambio de apuntadores	Crear `rose` · Declarar `turno` y `otra` · Escribir el ciclo con intercambio por variable temporal · Anunciar quién conserva energía	5	5
HU-11	Explicar intercambio de structs vs. de apuntadores	Dibujar cajas y flechas antes y después · Preparar la explicación oral	2	2
HU-12	Definir la estructura Cancion	Definir `Cancion` · Crear `discos[2]` en `main`	1	1
HU-13	Idol con canción opcional	Agregar el campo · Actualizar inicializadores con `NULL` · Recompilar y probar las etapas 1 a 3	2	2
HU-14	Asignar y quitar una canción	Escribir `asignar_cancion` · Asignar la misma canción a dos idols · Probar con `NULL`	3	3
		Total	**16**	16

Sprint Review: se acepta el sprint si…

- `colaboracion(&jennie, &jennie)` devuelve 0 sin cambiar nada y `colaboracion(NULL, &rose)` no detiene el programa.
- La dinámica de turnos termina y anuncia quién conserva energía.
- El equipo explica la diferencia entre intercambiar structs e intercambiar apuntadores.
- Una misma canción puede asignarse a dos integrantes y `asignar_cancion(&jennie, NULL)` se la quita.

Riesgos y consejos

- Verificar NULL **después** de usar `->` (ya es tarde).
- Intercambiar el contenido de los structs en lugar de las direcciones.
- Olvidar actualizar los inicializadores de `Idol` con `NULL` al agregar el campo `cancion`.

Etapa 3 · Dos apuntadores, NULL e intercambio

Concepto: funciones con dos apuntadores a struct, protección contra NULL y contra apuntadores repetidos, e intercambio de apuntadores (no de structs). **Producto:** `colaboracion` y una dinámica por turnos. **Sprint 2.**

HU-09 · Colaboración segura entre dos integrantes — Prioridad Alta · 3 pts

Como manager del grupo, **quiero** que dos integrantes distintas hagan una colaboración, **para** que ambas ganen popularidad sin que el programa falle si algo viene mal.

Criterios de aceptación

- Existe `int colaboracion(Idol *a, Idol *b)`.
- Si `a` es NULL, `b` es NULL o `a == b`, no modifica nada y devuelve 0.
- Si es válida: ambas suman popularidad (referencia: +10), ambas consumen energía con `limitar` (referencia: −15), se imprime un mensaje con sus nombres y devuelve 1.
- `colaboracion(&jennie, &jennie)` devuelve 0 y no cambia ningún dato.
- `colaboracion(NULL, &rose)` devuelve 0 sin detener el programa.
- La verificación de NULL ocurre **antes** de usar `->`.

Depende de: HU-04 · Sprint 2 · Entregable: `int colaboracion(Idol *a, Idol *b)`

HU-10 · Dinámica por turnos con intercambio de apuntadores — Prioridad Media · 5 pts

Como manager del grupo, **quiero** que dos integrantes ensayen por turnos hasta que una se quede sin energía, **para** ver quién resiste más.

Criterios de aceptación

- Existe una segunda idol (`rose`) y dos apuntadores, `turno` y `otra`.
- Mientras `esta_activa(turno) && esta_activa(otra)`, quien tiene el turno ensaya y los turnos se alternan.
- El cambio de turno se hace intercambiando los **apuntadores** con una variable temporal, no copiando structs.
- El ciclo termina y se anuncia en pantalla qué idol conserva energía.
- Ninguna energía queda fuera del rango 0 a `energia_max`.

Depende de: HU-05, HU-07 · Sprint 2 · Entregable: Ciclo `while` con `turno` y `otra`

HU-11 · Explicar intercambio de structs vs. de apuntadores — Prioridad Media · 2 pts

Como estudiante, **quiero** explicar la diferencia entre intercambiar dos structs e intercambiar las direcciones guardadas en dos apuntadores, **para** demostrar que entiendo qué se copia en cada caso.

Criterios de aceptación

- La explicación indica qué se copia en cada caso (todos los campos vs. una dirección) y qué cambia o no cambia en `jennie` y `rose`.
- Incluye un dibujo sencillo con las cajas `jennie`, `rose`, `turno` y `otra` y sus flechas, antes y después del intercambio.
- Se presenta oralmente en la Sprint Review.

Depende de: HU-10 · Sprint 2 · Entregable: Diagrama + explicación oral

Etapa 4 · Un struct que apunta a otro struct

Concepto: un campo que es apuntador a otro struct, NULL como “no hay nada asignado”, acceso encadenado `i->cancion->bonus` y un apuntador que apunta dentro de un arreglo. **Producto:** canciones asignadas, `popularidad_total` y la líder del `Grupo`. **Sprints 2 y 3.**

HU-12 · Definir la estructura Cancion	Prioridad Alta · 1 pts

Como manager del grupo, **quiero** registrar canciones con nombre, bono de popularidad y vigencia, **para** darles ventaja a las integrantes mientras la canción siga vigente.

Criterios de aceptación

- `Cancion` es un struct con `nombre` (30 caracteres), `bonus` y `vigencia` (presentaciones restantes; 0 significa vencida).
- En `main` existe `Cancion discos[2]` inicializado, por ejemplo `{{"Starlight", 10, 3}, {"Nova", 5, 6}}`.
- La definición de `Cancion` aparece antes de la de `Idol` en el archivo.

Depende de: HU-01 · Sprint 2 · Entregable: struct Cancion + `discos[2]`

HU-13 · Idol con canción opcional	Prioridad Alta · 2 pts

Como manager del grupo, **quiero** que una integrante pueda tener o no una canción asignada, **para** reflejar que algunas todavía no tienen una.

Criterios de aceptación

- `Idol` incluye el campo `Cancion *cancion;` al final.
- Todos los inicializadores de `Idol` terminan en `NULL`.
- Las funciones de las etapas anteriores (`mostrar`, `ensayar`, `colaboracion`…) siguen dando el mismo resultado.
- El programa compila sin advertencias.

Depende de: HU-12 · Sprint 2 · Entregable: Campo `Cancion *cancion;` en Idol

HU-14 · Asignar y quitar una canción	Prioridad Alta · 3 pts

Como manager del grupo, **quiero** asignar una canción a una integrante (incluso compartida) y poder quitársela, **para** administrar el repertorio del grupo.

Criterios de aceptación

- Existe `void asignar_cancion(Idol *i, Cancion *c)` que guarda en `i->cancion` la dirección recibida.
- La misma canción (`&discos[0]`) puede asignarse a dos integrantes distintas y ambas apuntan al mismo struct.
- `asignar_cancion(&jennie, NULL)` le quita la canción a la idol.
- Si `i` es NULL, la función no hace nada y no falla.

Depende de: HU-13 · Sprint 2 · Entregable: `void asignar_cancion(Idol *i, Cancion *c)`